

Clases que existen en el JAR

BubbleSort.class

MergeSortDemo.class

QuickSort.class

Sumarray.class

Operaciones sobre arreglos

```
public class BubbleSort {  
    public void bubbleSort(int[] arr) {  
        int n = arr.length;  
        for (int i = 0; i < n - 1; i++) {  
            boolean swapped = false;  
            for (int j = 0; j < n - 1 - i; j++) {  
                if (arr[j] > arr[j + 1]) {  
                    int temp = arr[j];  
                    arr[j] = arr[j + 1];  
                    arr[j + 1] = temp;  
                    swapped = true;  
                }  
            }  
            if (!swapped)  
                break;  
        }  
    }  
}
```

Lista de Clases



Algoritmos de ordenamiento

```
int n = arr.length;  
for (int i = 0; i < n - 1; i++) {
```

Pseudocódigo

L M M J V Pseudo código 1 1

Algoritmo Segundo Mayor y Menor (arreglo)

Si longitud del arreglo < 2 entonces
Mostrar "Se necesitan al menos dos elementos"
Terminar
Fin Si

mayor $\leftarrow -\infty$
Segundo Mayor $\leftarrow -\infty$
menor $\leftarrow +\infty$
Segundo menor $\leftarrow +\infty$

Para cada elemento x en el arreglo hacer

Si $x > \text{mayor}$ entonces
Segundo Mayor $\leftarrow \text{mayor}$
mayor $\leftarrow x$
Sino Si $x > \text{Segundo Mayor}$ y $x \neq \text{mayor}$
entonces Segundo Mayor $\leftarrow x$
Fin Si

Si $x < \text{menor}$ entonces
Segundo Menor $\leftarrow \text{menor}$
menor $\leftarrow x$
Sino Si $x < \text{Segundo Menor}$ y $x \neq \text{menor}$
entonces Segundo Menor $\leftarrow x$
Fin Si

L M M J V

Fin Para

Mostrar segundo Mayor

Mostrar segundo Menor

Fin Algoritmo

Explicación breve

El algoritmo funciona porque mantiene actualizadas cuatro variables: mayor, segundo mayor, menor y segundo menor mientras recorre el arreglo una sola vez.

Cada elemento es comparado y colocado en la posición correspondiente sin necesidad de ordenar el arreglo.

Análisis de complejidad

Tiempo:

$O(n)$

Espacio:

$O(1)$